

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

Determinarea automată a configurației tehnice a autoturismelor Dacia 1300/1310 folosind Deep Learning

Absolvent

Focșa Mihai-Luca

Coordonator științific

Conf.Dr. Mureșan Claudia

București, septembrie 2026

Rezumat

Lucrarea prezintă un sistem bazat pe Deep Learning pentru generarea automată a fișei tehnice a autoturismelor din gama Dacia 1300/1310, implementat cu PyTorch. Sistemul are o arhitectură pe două niveluri: primul nivel este un clasificator-router, care determină tipul imaginii (vedere frontală, vedere din spate sau compartimentul motor) și respinge imaginile pentru care încrederea este sub un prag configurabil. În funcție de rezultat, al doilea nivel interoghează automat clasificatorul specializat aferent: modelul (1300, 1310 CN3, CN4), tipul de caroserie (berlină sau break), respectiv tipul de alimentare (carburație sau injecție), iar predicțiile completează progresiv fișa tehnică a vehiculului. Toate clasificatoarele folosesc un backbone DINOv2 (Vision Transformer preantrenat self-supervised), complet înghețat, peste care se antrenează prin transfer learning câte un strat liniar, pe un set de date colectat și etichetat manual, în lipsa unui set de date public pentru aceste autoturisme. Performanța este măsurată prin validare încrucișată grupată pe vehicul, astfel încât evaluarea să reflecte comportamentul pe vehicule nevăzute la antrenare. Un astfel de sistem poate sprijini colecționarii și muzeele în identificarea configurației tehnice a vehiculelor istorice, etapă necesară, de exemplu, în evaluarea acestora.

Abstract

This thesis presents a Deep Learning system, implemented in PyTorch, that automatically generates the technical specification sheet of cars from the Dacia 1300/1310 family. The system uses a two-level architecture: the first level is a router-classifier, which determines the type of the input image (front view, rear view, or engine bay) and rejects images whose confidence score falls below a configurable threshold. Depending on the result, the second level automatically queries the corresponding specialized classifier: the model variant (1300, 1310 CN3 and CN4), the body style (sedan or station wagon), or the fuel system (carburetor or fuel injection), and the predictions progressively fill in the vehicle's technical sheet. All classifiers share a frozen DINOv2 backbone (a self-supervised pretrained Vision Transformer), on top of which a single linear layer per task is trained via transfer learning, on a dataset that was collected and labeled manually, as no public dataset exists for these vehicles. Performance is measured with vehicle-grouped cross-validation, so that the evaluation reflects behaviour on vehicles unseen during training. Such a system can assist collectors and museums in identifying the technical configuration of historical vehicles, a necessary step, for instance, in their appraisal.

Cuprins

1 Introducere **7**

1.1 Motivație 7

1.2 Definiție formală și dificultăți 7

1.3 Obiective 8

2 Preliminarii **9**

2.1 Rețelele neuronale convoluționale 9

2.2 Vision Transformers și DINOv2 9

2.3 Transfer learning și linear probing 9

2.4 Fine-grained classification 10

3 Setul de date **11**

3.1 Colectare și etichetare 11

3.2 Exemple de imagini 11

3.3 Gruparea imaginilor pe vehicul 12

3.4 Preprocesare 13

4 Arhitectura sistemului **14**

4.1 Privire generală 14

4.2 Router-classifier 14

4.3 Clasificatori specializați 15

4.4 Mecanismul de respingere 15

5 Antrenarea modelelor **16**

5.1 Hardware 16

5.2 Transfer Learning 16

5.3 Augmentarea datelor 16

5.4 Hiperparametri 17
5.5 Optimizare 17

6 Implementare **18**
6.1 Tehnologii folosite 18
6.1.1 Python 18
6.1.2 PyTorch și torchvision 18
6.1.3 scikit-learn 18
6.1.4 Tkinter 18
6.1.5 Matplotlib 18
6.1.6 Pillow 19
6.2 Organizarea proiectului 19
6.3 Interfața grafică 19
6.4 Dificultăți întâmpinate 20

7 Rezultate **21**
7.1 Metodologia de evaluare 21
7.2 Curbe de antrenare 21
7.3 Acuratețe pe clasificatori 22
7.4 Discuție și limitări 22

8 Concluzii **24**

Bibliografie **25**

A Exemple de imagini din setul de date **28**

B Fragmente de cod **30**

C Curbe de antrenare **32**

D Reguli și paginile aplicației **33**

1 Introducere

1.1 Motivație

Autoturismele Dacia 1300 și 1310 au fost cele mai răspândite autoturisme produse în România, producția începând în 1969 și terminându-se în 2004. De-a lungul acestei perioade, modelul a cunoscut diverse variante: caroserii diferite (berlină, break și pickup), versiuni pentru export, evoluții estetice și motorizări diferite (carburație mono/dublu corp, injecție).

Pentru colecționari sau muzee, este imperativă stabilirea unei fișe tehnice a mașinii, iar acest lucru poate să devină complicat întrucât anumite diferențe sunt subtile și doar un cunoscător poate stabili cu certitudine configurația tehnică a autoturismului. Astfel, automatizarea acestui proces prin vederea artificială constituie problema centrală a lucrării.

1.2 Definiție formală și dificultăți

Din punct de vedere formal, definim:

$$\phi(x) = y, \quad \text{cu } x \in R^{3\times H\times W}, \quad \text{și } y = (y_{\text{caroserie}}, y_{\text{model}}, y_{\text{motor}})$$

Notăm faptul că tipul de motor este vizibil doar din compartimentul motor, jantele se văd doar dintr-un unghi lateral și tipul de model se poate decide doar când avem un cadru frontal. Astfel, o singură imagine x conține rareori informație (features) suficientă ca să putem completa integral tripletul y. Mai mult de atât, problema este, în mare parte, de tip fine-grained classification, deoarece fiecare clasă diferă prin detalii subtile (într-adevăr, fiecare model este o evoluție a vehiculului de bază, deci există similarități).

1.3 Obiective

Pentru întocmirea unui software robust, stabilim următoarele obiective:

1. Elaborarea unei arhitecturi în care un router decide ce tip de imagine este, iar după interogăm modelul aferent. Alternativ, putem interoga manual un model.

2. Antrenarea unor clasificatoare specializate pe câte o caracteristică tehnică cu transfer learning pe un set de date relativ redus.

3. Distingerea și respingerea imaginilor care nu se încadrează în nicio categorie.

4. Evaluarea corectă a clasificatoarelor, ținând cont că același vehicul apare în mai multe imagini.

5. Edificarea unei interfețe grafice pentru a putea executa operațiunile și vizualiza rezultatele.

2 Preliminarii

2.1 Rețelele neuronale convoluționale

Rețelele neuronale convoluționale (CNN) sunt, în general, standardul pentru un proiect care implică clasificarea imaginilor [7]. Ele aplică filtre convoluționale învățate care extrag trăsături (features) simple și complexe, cum ar fi margini, texturi, forme și obiecte. Antrenarea de la zero a unui CNN destul de complex pentru nevoile acestui proiect nu este realizabil deoarece am avea nevoie de sute de mii, poate chiar milioane de imagini.

2.2 Vision Transformers și DINOv2

Pe lângă CNN-uri, o a doua familie de arhitecturi folosită pe scară largă la clasificarea imaginilor este Vision Transformer (ViT) [3]. Un ViT împarte imaginea în patch-uri (în cazul nostru de 14×14 pixeli) și le procesează cu mecanismul de self-attention, astfel încât fiecare patch ține cont de întreaga imagine, nu doar de vecinătatea lui.

În această lucrare folosim DINOv2 [9], un ViT preantrenat self-supervised (fără etichete) pe un volum foarte mare de imagini. Avantajul practic: embeddings-urile produse de el (vectori de 384 de dimensiuni pentru varianta ViT-S/14, aprox. 21 de milioane de parametri) descriu suficient de bine conținutul vizual încât backbone-ul poate rămâne complet înghețat, iar noi antrenăm doar un strat liniar deasupra. La volumul nostru de date (sute de imagini per sarcină), asta reduce drastic riscul de overfitting față de adaptarea a milioane de parametri.

2.3 Transfer learning și linear probing

Transfer learning înseamnă reutilizarea unui model antrenat pe o sarcină inițială pentru a fi folosit la o sarcină diferită [15]. Există două variante uzuale:

- Linear probing (feature extraction): dăm freeze la backbone și antrenăm doar stratul final de clasificare. Se antrenează foarte puțini parametri, deci riscul de overfitting este mic.
- Fine-tuning: dăm unfreeze la o parte din backbone, adaptându-l la sarcina noastră, cu un learning rate relativ mic (altfel, am distruge trăsăturile preînvățate). Este mai puternic, dar riscant când datele sunt puține.

Am ales prima variantă pentru toate clasificatoarele: antrenăm câte un singur strat liniar (sub 1200 de parametri) peste embeddings-urile DINOv2. Pe lângă riscul mic de overfitting, antrenarea durează câteva secunde, ceea ce va face posibilă evaluarea prin validare încrucișată din capitolul Rezultate.

2.4 Fine-grained classification

Distincția dintre modelele de Dacia, în special variațiile CN de 1310, ține de detalii subtile, cum ar fi forma grilei, aranjarea și culoarea sticlei/plasticului de la faruri sau ornamente. Acest tip de configurație, similară cu seturi de date precum CUB-200 (specii de păsări) [14] sau Stanford Cars [6], necesită o abordare specială. În cazul nostru, sarcina de a surprinde detaliile fine cade pe embeddings-urile DINOv2, preantrenate exact pentru a diferenția conținut vizual asemănător; toate imaginile sunt redimensionate la 224×224 (multiplu al dimensiunii de patch, 14).

3 Setul de date

3.1 Colectare și etichetare

Nu există seturi de date publice pentru autoturismele din gama Dacia 1300/1310, deci întregul dataset a fost colectat și etichetat manual, din forumuri de pasionați, platforme de vânzări, și poze făcute la rarele autovehicule care încă pot fi zărite pe străzi. Etichetarea a fost realizată pe foldere, fiecare folder reprezentând o clasă. Distribuția finală este rezumată în Tabela 3.1.

3.2 Exemple de imagini

Exemple de imagini din setul de date, pentru fiecare sarcină și clasă, sunt prezentate în Anexa A (Figurile A.1–A.3).

Tabela 3.1: Distribuția imaginilor pe sarcini și clase.

Sarcină	**Clasă**	**Nr. imagini**
Caroserie	berlină	130
	break	134
	total	*264*
Generație / variantă	1300	44
(partea frontală)	CN3	45
	CN4	47
	total	*136*
Motor	carburație	46
	injecție	66
	total	*112*
Router	față	136
	motor	112
	spate	264
	total	*512*

3.3 Gruparea imaginilor pe vehicul

Multe imagini provin din același anunț sau de la același vehicul fotografiat din mai multe unghiuri. Acest lucru contează pentru evaluare: dacă poze diferite ale aceleiași mașini ajung și în antrenare și în validare, modelul recunoaște mașina deja văzută în loc să generalizeze (revenim asupra acestui aspect în capitolul Rezultate). De aceea, fiecare imagine a primit un identificator de vehicul, codificat în numele fișierului (prefixul `car<ID>__`).

Gruparea manuală a sute de imagini ar fi fost laborioasă, așa că am folosit un procedeu semi-automat: pozele aceluiași vehicul au embeddings DINOv2 foarte apropiate, deci am propus automat grupe pe baza similarității cosinus (peste un prag de 0,90), apoi am verificat și

corectat manual propunerile într-o pagină HTML cu thumbnails. În final, cele 512 imagini aparțin la 417 vehicule distincte.

3.4 Preprocesare

Toate modelele urmează aceleași etape de preprocesare:

1. **PadToSquare** (Cod B.1, Anexa B): completăm imaginea cu pixeli negri astfel încât să fie în format pătrat deoarece vrem să evităm problemele de distorsiune (este foarte important în categoria caroseriilor de tip break, întrucât este mai lungă și forma geometrică a șasiului este foarte probabilă să se modifice când redimensionăm).

2. **Redimensionare**: 224×224 pentru toate clasificatoarele (multiplu al dimensiunii patch-ului ViT-S/14)

3. **Normalizare**: folosim media și deviația standard ImageNet [2] ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$)

4 Arhitectura sistemului

4.1 Privire generală

În loc de un clasificator de tip monolit care să analizeze toate atributele, procesul de întocmire a fișei tehnice este cel din Figura 4.1:

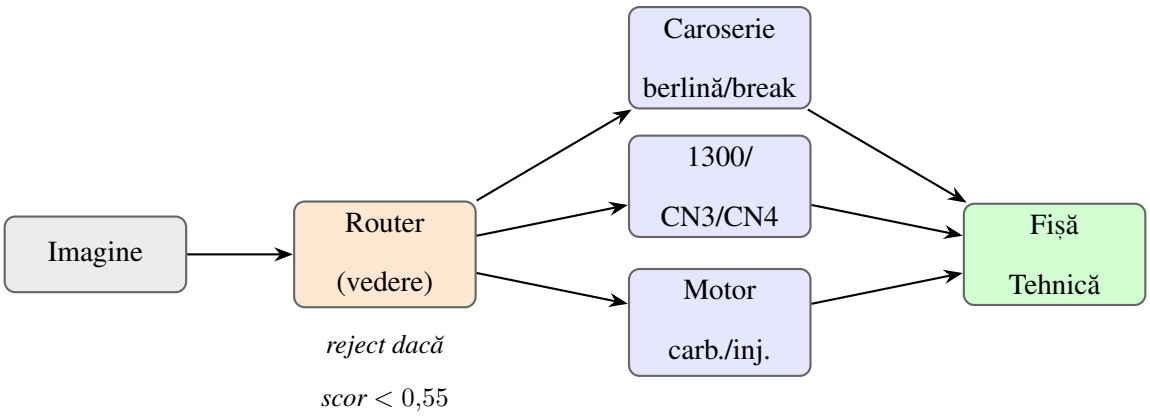


Figura 4.1: Arhitectura *router* → *specialiști*. Routerul decide natura imagini și o trimite doar modelelor care pot clasifica imaginea. Un prag de respingere filtrează imaginile nesigure, setat inițial la 0.55 dar modificabil din GUI.

4.2 Router-classifier

Routerul are aceeași arhitectură ca specialiștii (un strat liniar peste embeddings DINOv2, detaliată în capitolul următor) și clasifică imaginea în una din următoarele clase: față, motor, spate. Acesta se bazează pe contextul global al imaginii, nu pe detalii. Datele de antrenare ale routerului sunt construite automat din folderele cu datele pentru clasificatoarele specializate de către scriptul populateRouter.py: imaginile din `model/*` devin clasa `fata`, cele din `caroserie/*` devin `spate`, iar cele din `motor/*` devin `motor` (Cod B.2, Anexa B).

4.3 Clasificatori specializați

Dispunem de 3 modele specializate pe câte o parte a fișei tehnice:

- **Caroserie**: berlină vs break (2 clase, clasificator binar)
- **Model**: 1300/CN3/CN4 (3 clase)
- **Motor**: carburație vs injecție (2 clase, clasificator binar)

Am ales să le separăm deoarece fiecare specialist vede doar tipul de imagine relevant pentru el (de exemplu, clasificatorul de caroserie primește doar vederi din spate), iar o caracteristică nouă în fișă înseamnă doar un clasificator nou, fără a le reantrena pe celelalte.

4.4 Mecanismul de respingere

La inferență, routerul aplică softmax pe outputuri și compară scorul maxim cu un prag $\tau = 0,55$, modificabil din setările GUI. Dacă scorul maxim este sub prag, imaginea este respinsă și nu interogăm niciun clasificator (Cod B.3, Anexa B). Practic, preferăm să nu dăm niciun răspuns pentru o poză pentru care routerul nu este sigur.

5 Antrenarea modelelor

5.1 Hardware

Toate experimentele au fost rulate local, pe un laptop personal cu placă video NVIDIA RTX 3050 (4 GB VRAM). Această limitare a cântărit în alegerea unui backbone compact și înghețat: embeddings-urile DINOv2 se extrag o singură dată pentru tot setul (câteva minute), iar antrenarea unui strat liniar durează câteva secunde. Tocmai această viteză a făcut posibilă validarea încrucișată cu 5 fold-uri din capitolul Rezultate.

5.2 Transfer Learning

Toate clasificatoarele au aceeași structură: backbone-ul DINOv2 ViT-S/14 [9], complet înghețat, produce pentru fiecare imagine un embedding de 384 de dimensiuni (normalizat L2), iar peste el antrenăm un singur strat liniar cu numărul de clase al sarcinii. Embeddings-urile se calculează o singură dată și se refolosesc la fiecare antrenare (Cod B.4, Anexa B).

5.3 Augmentarea datelor

Backbone-ul fiind înghețat, augmentările clasice pe pixeli (crop aleator, rotație, jittering de culoare) nu mai au rolul pe care îl au la fine-tuning, pentru că filtrele nu se mai adaptează. Am păstrat o singură augmentare, oglindirea orizontală, aplicată la nivel de embeddings: pentru fiecare imagine calculăm și embedding-ul variantei oglindite, ceea ce dublează datele văzute de stratul liniar.

Aceeași idee este folosită și la inferență, ca test-time augmentation (TTA): predicția finală este media probabilităților obținute pe imaginea originală și pe cea oglindită orizontal.

5.4 Hiperparametri

Spre deosebire de un fine-tuning clasic, unde fiecare sarcină cere rezoluție și augmentări proprii, aici toate cele patru clasificatoare folosesc exact aceeași configurație:

Tabela 5.1: Configurația de antrenare, comună tuturor clasificatoarelor.

Parametru	**Valoare**
Rezoluție de intrare	224×224
Dimensiune embedding	384
Parametri antrenați	sub 1200 (un strat liniar)
Batch	full-batch (tot setul odată)
Epoci	200
Optimizator	Adam, LR 10^{-3}, weight decay 10^{-4}
Label smoothing	0,1
Ponderi de clasă	invers proporționale cu frecvența

5.5 Optimizare

Toate modelele folosesc optimizatorul Adam [5] și funcția de loss CrossEntropyLoss, cu două ajustări pentru un set mic și dezechilibrat: label smoothing de 0,1 și ponderi de clasă invers proporționale cu numărul de exemple (de exemplu, la motor: 46 imagini cu carburație vs 66 cu injecție). Pentru că lucrăm direct pe embeddings, antrenarea se face full-batch, deci nu mai este nevoie de scheduler și nici de selecția celei mai bune epoci: loss-ul scade monoton, iar acuratețea de validare este doar monitorizată, nu folosită la alegerea modelului. Evaluarea se face prin validare încrucișată grupată pe vehicul (capitolul Rezultate), iar modelul final livrat aplicației se antrenează pe toate datele, cu aceeași configurație.

6 Implementare

6.1 Tehnologii folosite

6.1.1 Python

Python [13] este limbajul în care este scris întregul proiect, ales pentru bibliotecile de machine learning și de procesare a imaginilor pe care se sprijină elementele de mai jos.

6.1.2 PyTorch și torchvision

PyTorch [10] este framework-ul de deep learning folosit pentru a defini, antrena și salva clasificatoarele. Backbone-ul DINOv2 se încarcă prin torch.hub, iar din torchvision [8] am preluat transformările aplicate imaginilor (redimensionare, normalizare).

6.1.3 scikit-learn

Scikit-learn [11] este folosit la evaluare: StratifiedGroupKFold implementează validarea încrucișată stratificată pe clase și grupată pe vehicul, iar confusion_matrix stă la baza matricelor de confuzie din capitolul Rezultate.

6.1.4 Tkinter

Tkinter [12] este biblioteca grafică standard din Python, disponibilă fără instalări suplimentare. Pe ea este construită interfața desktop, cu cele trei tab-uri (Auto, Manual, Settings) și afișarea progresivă a fișei tehnice.

6.1.5 Matplotlib

Matplotlib [4] este folosit pentru reprezentări grafice. Cu el am generat curbele de antrenare și graficele de evaluare din capitolul Rezultate.

6.1.6 Pillow

Pillow [1] se ocupă de citirea și manipularea imaginilor. Operează direct pe obiectele imagine în etapa de preprocesare, inclusiv în pasul PadToSquare care completează imaginea până la format pătrat.

6.2 Organizarea proiectului

- **train/**: datele de antrenare, organizate pe clase
- **scripts/**: scripturile de antrenare, evaluare și grupare pe vehicul
- **models/**: aici se regăsesc modelele
- **graphics/**: curbele de antrenare și graficele de evaluare
- **app/**: interfața grafică și motorul de inferență

6.3 Interfața grafică

Interfața este o aplicație desktop în tkinter care, din poze cu un autoturism din gama Dacia 1300/1310, completează automat o fișă tehnică. Ea este împărțită în două: în stânga, sunt trei tab-uri (**Auto**: rulăm routerul pe imagine, iar aplicația decide singură ce surprinde poza și cheamă automat modelele specializate relevante; **Manual**: alegem noi modelul, apoi îl interogăm și vedem distribuția completă de probabilități pe clase; **Settings**: toate setările aplicației, adică device (auto/cpu/cuda), calea către modele, pragul de respingere, dar și lista modelelor care pot fi interogate).

În dreapta se află fișa tehnică, care se umple cu detaliile tehnice identificate de model, fiecare însoțit de scorul de încredere. Pe lângă valorile prezise direct, aplicația aplică și reguli de derivare specifice domeniului (de exemplu, din varianta `1300` deduce cilindreea de `1289` cm^3). Dacă scorul routerului este sub pragul de respingere, imaginea este considerată neclară sau irelevantă, iar fișa nu se completează. Notăm că backbone-ul DINOv2 este comun tuturor clasificatoarelor și se încarcă o singură dată, modelele sunt păstrate într-un cache, iar inferența

rulează pe un fir de execuție separat ca să nu înghețe interfața. După acest proces, datele se pot exporta într-un fișier text.

6.4 Dificultăți întâmpinate

Cele mai mari dificultăți au fost legate de date. Colectarea a fost laborioasă: imaginile provin din surse foarte diferite, deci variază mult ca unghi, iluminare și rezoluție, iar o parte dintre ele au trebuit eliminate fiindcă erau neclare sau surprindeau vehiculul doar parțial. Pentru variantele mai rare de 1310, numărul de exemple găsite a fost foarte mic, ceea ce a limitat dimensiunea finală a setului. A doua dificultate a fost evaluarea corectă: același vehicul apare în mai multe imagini, deci a trebuit să grupez cele 512 imagini pe vehicul (procedeul semi-automat din Secțiunea 3.3) și să trec de la un split simplu la validarea încrucișată grupată, tratată pe larg în capitolul Rezultate.

7 Rezultate

7.1 Metodologia de evaluare

O evaluare cu split aleator la nivel de imagine (80/20) supraestimează performanța pe acest set de date: același vehicul apare în mai multe poze, iar dacă unele ajung în antrenare și altele în validare, modelul recunoaște mașina deja văzută în loc să generalizeze. În experimentele noastre inițiale, exact această scurgere de date ducea la acurateți de validare de 100% la mai mulți clasificatori — un semn de evaluare greșită, nu de model perfect.

Din acest motiv, toate rezultatele raportate aici folosesc validare încrucișată cu 5 fold-uri, stratificată pe clase și grupată pe vehicul (*StratifiedGroupKFold*): toate imaginile unui vehicul (Secțiunea 3.3) cad în același fold, deci acuratețea măsoară comportamentul pe vehicule nevăzute la antrenare — exact scenariul real de utilizare. Raportăm media și deviația standard a acurateței peste fold-uri.

7.2 Curbe de antrenare

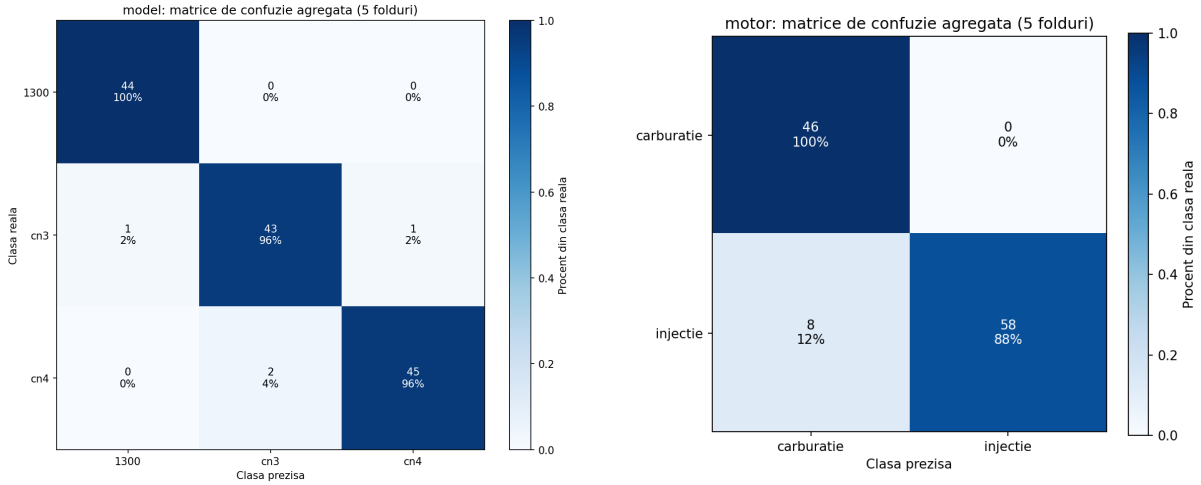
Figurile C.1 și C.2 (Anexa C) prezintă evoluția loss-ului de antrenare al stratului liniar și a acurateței de validare, cu câte o curbă pentru fiecare fold, pentru router, respectiv pentru clasificatorul de caroserie. Loss-ul scade monoton, acuratețea se stabilizează în prima parte a antrenării, iar la caroserie se văd diferențe vizibile între fold-uri. Reamintim că acuratețea de validare este doar monitorizată; nu se face selecție de epocă pe baza ei.

7.3 Acurateţe pe clasificatori

Tabela 7.1: Acurateţea validării încrucişate grupate pe vehicul (media $\pm$ deviaţia standard peste 5 fold-uri).

Clasificator	Nr. clase	Imagini / vehicule	Acurateţe CV
Router (faţă/motor/spate)	3	512 / 417	$98{,}9\% \pm 0{,}7\%$
Caroserie (berlină/break)	2	264 / 220	$91{,}6\% \pm 2{,}9\%$
Variantă (1300/CN3/CN4)	3	136 / 100	$96{,}7\% \pm 3{,}3\%$
Motor (carburaţie/injecţie)	2	112 / 97	$93{,}6\% \pm 10{,}5\%$

Figura 7.1 prezintă matricele de confuzie agregate peste cele 5 fold-uri pentru cei doi clasificatori fine-grained. La variantă, clasa 1300 se separă perfect (recall 100%), iar puţinele confuzii apar între CN3 şi CN4, variantele cele mai asemănătoare vizual. La motor, toate erorile sunt într-o singură direcţie: injecţii clasificate drept carburaţie (recall injecţie 87,9%).



(a) Variantă (1300/CN3/CN4)

(b) Motor (carburaţie/injecţie)

Figura 7.1: Matricele de confuzie agregate peste cele 5 fold-uri (rând = clasa reală).

7.4 Discuţie şi limitări

Ierarhia rezultatelor confirmă intuiţiile despre dificultatea sarcinilor. Router-ul atinge cea mai bună acurateţe, 98,9%, deoarece distincţia între o vedere frontală, una din spate şi comparti-

mentul motor se bazează pe contextul global al imaginii, nu pe detalii fine. Clasificatorul de variantă ajunge la 96,7%, clasele fiind vizual distincte (faruri rotunde și grilă cromată la 1300, faruri dreptunghiulare la CN3, grilă redesenată la CN4). Caroseria, în schimb, rămâne la 91,6%: multe vederi din spate sunt fotografiate din unghiuri în care diferența berlină/break se vede greu.

Rezultatul de la motor trebuie citit cu prudență: media de 93,6% vine cu o deviație standard de 10,5 puncte procentuale, pentru că un fold coboară la 73,1% în timp ce celelalte patru sunt aproape perfecte. Cu doar 112 imagini de la 97 de vehicule, fiecare fold de validare are în jur de 20 de imagini, deci câteva greșeli schimbă acuratețea cu multe puncte procentuale. Îl considerăm un rezultat preliminar, iar colectarea de date suplimentare pentru această sarcină este principala prioritate.

Mai notăm trei limitări. Prima: metrica are rezoluție mică în general — la fold-uri de 20–115 imagini, diferențele de 1–2 puncte procentuale între fold-uri sunt zgomot, motiv pentru care raportăm mereu media cu deviația standard, nu o valoare unică. A doua: în setul de variantă, pozele clasei 1300 au rezoluții mai mari (în jur de 850 de pixeli lățime) decât cele din CN3/CN4 (300–500 de pixeli); redimensionarea la 224×224 atenuează diferența, dar o corelație reziduală între calitatea imaginii și clasă nu poate fi exclusă. A treia: gruparea pe vehicul este semi-automată, deci dacă același autoturism apare în două anunțuri diferite sub identificatori diferiți, o mică scurgere de date poate persista. De asemenea, întreaga evaluare folosește un singur seed; repetarea validării încrucișate cu mai multe seed-uri ar consolida estimările.

8 Concluzii

Lucrarea a pornit de la o problemă concretă, determinarea automată a configurației tehnice a autoturismelor Dacia 1300/1310, și a condus la un sistem în care un router dirijează imaginea către clasificatorul potrivit, iar predicțiile completează progresiv fișa tehnică. Toate clasificatoarele împart același backbone DINOv2 înghețat, peste care se antrenează câte un strat liniar, iar performanța a fost măsurată prin validare încrucișată grupată pe vehicul, adică pe vehicule nevăzute la antrenare. Rezultatele obținute sunt încurajatoare, principalele limitări ținând de volumul redus și de modul de colectare a datelor.

Sistemul poate fi îmbunătățit din mai multe perspective. În primul rând, extinderea la alte autoturisme, dincolo de gama 1300/1310, ceea ce ar transforma aplicația într-un instrument mai general pentru vehicule de epocă. O a doua direcție este adăugarea unui clasificator pentru culorile de epocă (de exemplu Roșu 28, Portocaliu 331 sau Roșu 25), care ar completa fișa tehnică cu nuanța originală a vehiculului. În final, colectarea de date suplimentare pentru clasificatorul de motorizare, al cărui rezultat este deocamdată instabil între fold-uri, și repetarea validării încrucișate cu mai multe seed-uri ar consolida estimările raportate.

Bibliografie

[1] Alex Clark și Pillow Contributors, *Pillow: The Python Imaging Library*, https://python-pillow.org, Accesat: 2026, 2024.

[2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li și Li Fei-Fei, „ImageNet: A Large-Scale Hierarchical Image Database", în *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255, DOI: 10.1109/CVPR.2009.5206848.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit și Neil Houlsby, „An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", în *International Conference on Learning Representations (ICLR)*, 2021.

[4] John D. Hunter, „Matplotlib: A 2D Graphics Environment", în *Computing in Science & Engineering* 9.3 (2007), pp. 90–95, DOI: 10.1109/MCSE.2007.55.

[5] Diederik P. Kingma și Jimmy Ba, „Adam: A Method for Stochastic Optimization", în *International Conference on Learning Representations (ICLR)*, 2015.

[6] Jonathan Krause, Michael Stark, Jia Deng și Li Fei-Fei, „3D Object Representations for Fine-Grained Categorization", în *Proceedings of the IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2013, pp. 554–561, DOI: 10.1109/ICCVW.2013.77.

[7] Alex Krizhevsky, Ilya Sutskever și Geoffrey E. Hinton, „ImageNet Classification with Deep Convolutional Neural Networks", în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[8] Sébastien Marcel și Yann Rodriguez, „Torchvision the Machine-Vision Package of Torch”, în *Proceedings of the 18th ACM International Conference on Multimedia*, 2010, pp. 1485–1488, DOI: 10.1145/1873951.1874254.

[9] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin și Piotr Bojanowski, „DINOv2: Learning Robust Visual Features without Supervision”, în *Transactions on Machine Learning Research (TMLR)* (2024).

[10] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai și Soumith Chintala, „PyTorch: An Imperative Style, High-Performance Deep Learning Library”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot și Édouard Duchesnay, „Scikit-learn: Machine Learning in Python”, în *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[12] Python Software Foundation, *tkinter — Python interface to Tcl/Tk*, https://docs.python.org/3/library/tkinter.html, Accesat: 2026, 2024.

[13] Guido van Rossum și Fred L. Drake, *Python 3 Reference Manual*, Scotts Valley, CA: CreateSpace, 2009.

[14] Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona și Serge Belongie, *The Caltech-UCSD Birds-200-2011 Dataset*, rap. teh. CNS-TR-2011-001, California Institute of Technology, 2011.

[15] Jason Yosinski, Jeff Clune, Yoshua Bengio și Hod Lipson, „How Transferable Are Features in Deep Neural Networks?”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

Anexa A Exemple de imagini din setul de date



(a) Dacia 1300



(b) Dacia 1310 CN3



(c) Dacia 1310 CN4

Figura A.1: Exemple de imagini din setul de date, câte una pentru fiecare variantă ce poate fi identificată de clasificatorul pentru modele.



(a) Spate Dacia Berlină



(b) Spate Dacia break

Figura A.2: Cele două tipuri de caroserie pe care le poate identifica clasificatorul binar pentru șasiuri.



(a) Modul de injecție



(b) Carburator

Figura A.3: Cele două tipuri de alimentație identificate de clasificatorul pentru motorizări.

Anexa B Fragmente de cod

```python
class PadToSquare:
    def __call__(self, img):
        w, h = img.size
        max_wh = max(w, h)
        padding = ((max_wh - w) // 2, (max_wh - h) // 2,
                   max_wh - w - (max_wh - w) // 2,
                   max_wh - h - (max_wh - h) // 2)
        return F.pad(img, padding, 0, 'constant')
```

Cod B.1: Preprocesarea PadToSquare, identică în antrenare și inferență.

```python
MAPPING = {
    "fata":    ["model/1300", "model/cn3", "model/cn4"],
    "spate":   ["caroserie/berlina", "caroserie/break"],
    "motor":   ["motor/carburatie", "motor/injectie"],
}
```

Cod B.2: Maparea sursă → clasă de router.

```python
top_class, top_prob = scores[0]
if top_prob < REJECT_THRESHOLD:
    print(f"REJECT | top={top_class} ({top_prob*100:.2f}%)")
    return
specialisti = SPECIALIST_MAP.get(top_class, [])
print(f"DISPATCH -> {top_class} | specialisti: {specialisti}")
```

Cod B.3: Logica de respingere.

```python
probe = nn.Linear(384, n_classes)
opt = torch.optim.Adam(probe.parameters(), lr=1e-3,
```

```
                          weight_decay=1e-4)
crit = nn.CrossEntropyLoss(weight=class_weights,
                           label_smoothing=0.1)
for epoch in range(200):
    opt.zero_grad()
    loss = crit(probe(X_train), y_train)  # X_train = embeddings
    loss.backward()
    opt.step()
```

Cod B.4: Antrenarea stratului liniar peste embeddings (full-batch).

Anexa C Curbe de antrenare

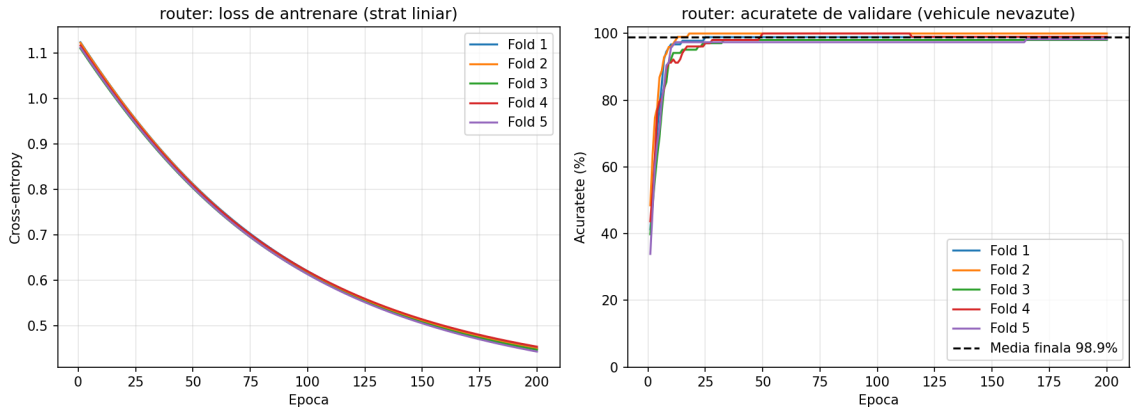


Figura C.1: Router-ul (față/motor/spate), 200 de epoci, câte o curbă per fold. Acuratețea de validare se stabilizează rapid, în jurul mediei de 98,9%.

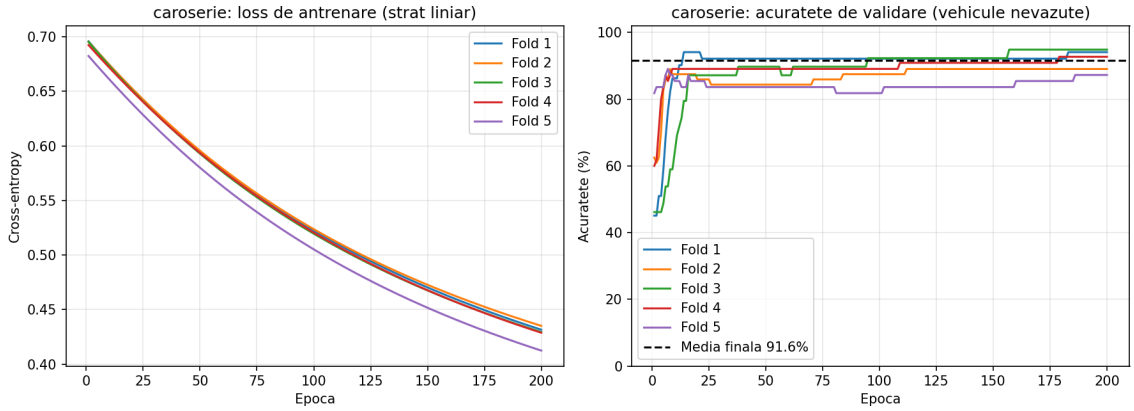


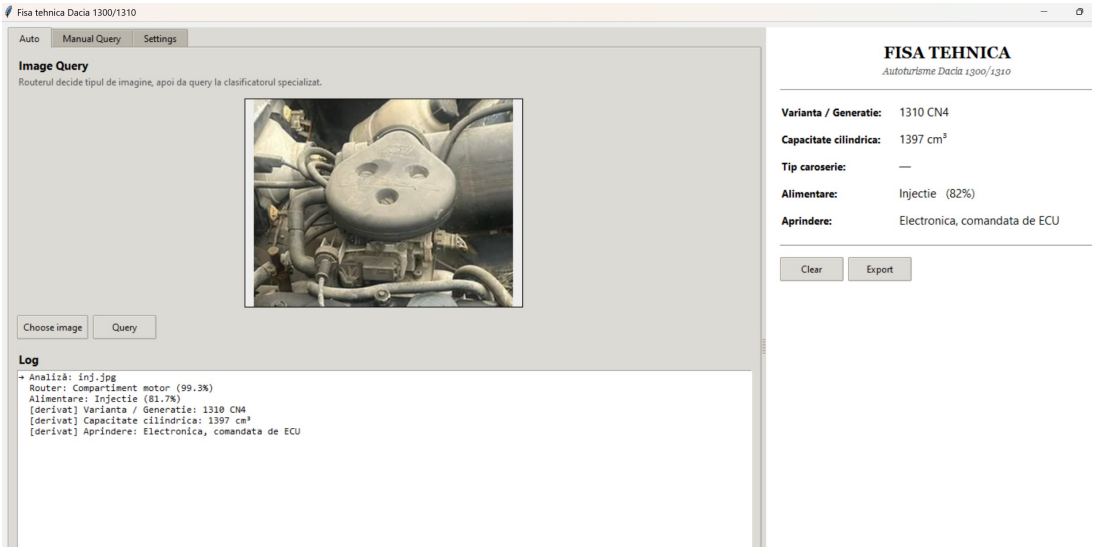
Figura C.2: Clasificatorul de caroserie (berlină/break). Acuratețea de validare se stabilizează în jurul valorii de 91–92%, cu diferențe vizibile între fold-uri.

Anexa D Reguli și paginile aplicației

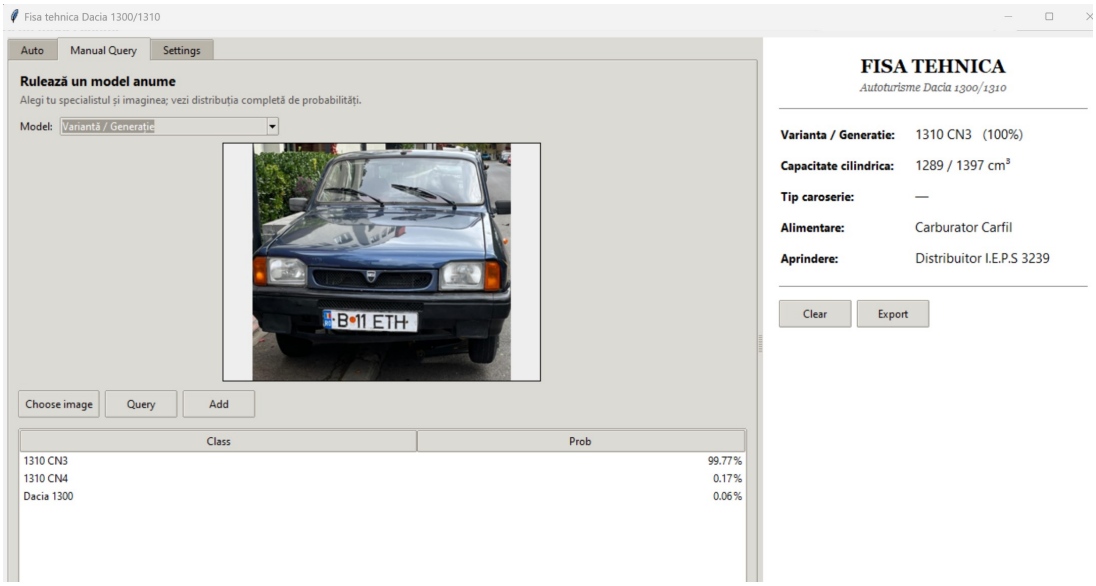
Pe lângă atributele prezise direct de clasificatori, aplicația completează fișa tehnică și cu valori derivate determinist din varianta detectată, conform regulilor din Tabela D.1.

Tabela D.1: Reguli folosite de aplicație.

Varianta detectată	**Capacitate cilindrică**	**Alimentare**	**Aprindere**
Dacia 1300	1289 cm^3	Carfil 32 IRM / Solex 32 EISA-3	Distribuitor Ducellier 4224
Dacia 1310 CN3	1289/1397 cm^3	Carfil 32 IRM	Distribuitor I.E.P.S 3239
Dacia 1310 CN4	1397 cm^3	Injecție monopunct Bosch	Electronică, comandată de ECU



(a) Modul **Auto**: routerul decide tipul imaginii și completează fișa.



(b) Modul **Manual**: distribuția de probabilități pe clase.

Figura D.1: Capturi de ecran din aplicație.

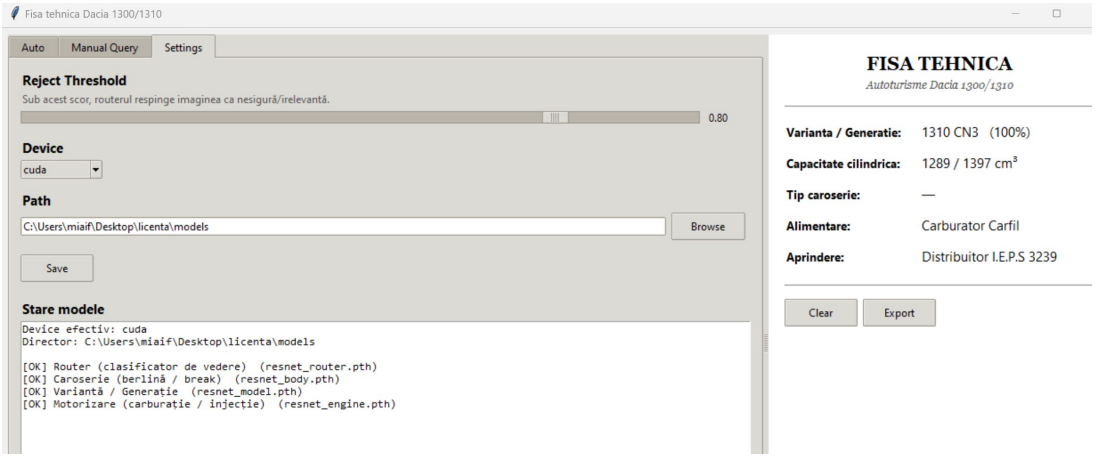


Figura D.2: Tab-ul **Settings**: configurarea pragului și a căilor.